Algoritmo Evolutivo: Differential Evolution para Otimização

Davi S. Mattos[119133049]

Universidade Federal do Rio de Janeiro, Rio de Janeiro - RJ, Brazil
davi.mattos@ic.ufrj.br

Abstract. Este trabalho apresenta um estudo comparativo de diferentes estratégias de Differential Evolution (DE), um algoritmo evolutivo baseado em população para otimização contínua. Implementamos cinco estratégias distintas (DE/Rand/1/bin, DE/Rand/2/bin, DE/Best/1/bin, DE/Best/2/bin e DE/Current-to-Best/1/bin) e as testamos na função de Rosenbrock bidimensional. Através de 30 execuções independentes com 10.000 avaliações de função cada, observamos que a estratégia DE/Best/1/bin alcançou o melhor desempenho geral, atingindo valores de fitness final da ordem de 10^{-19}. Os resultados demonstram a importância da estratégia de mutação na eficiência do algoritmo e validam a implementação através de comparação com a biblioteca pymoo.

Keywords: Differential Evolution · Otimização Contínua · Algoritmo Evolutivo · Função de Rosenbrock

1 Introdução

A otimização de funções contínuas é um problema fundamental em ciência e engenharia, com aplicações em machine learning, design de circuitos, processamento de imagem e muitos outros domínios. Os métodos tradicionais baseados em gradiente (como gradient descent) funcionam bem para funções unimodais diferenciáveis, mas enfrentam dificuldades em cenários com múltiplos ótimos locais, descontinuidades ou ausência de informação de gradiente [1].

Os algoritmos evolutivos representam uma classe alternativa de métodos de otimização inspirados em processos biológicos. Entre eles, o Differential Evolution (DE) destaca-se por sua simplicidade, eficiência e robustez [2]. O DE utiliza operações como mutação diferencial e cruzamento para explorar o espaço de busca, mantendo uma população de candidatos e promovendo os mais promissores.

A função de Rosenbrock é um benchmark clássico para algoritmos de otimização contínua. Sua topologia desafiadora, com um vale estreito e alongado, torna-a especialmente adequada para avaliar a capacidade de exploração-explotação de diferentes estratégias [3].

Este relatório apresenta uma investigação empírica de cinco estratégias de DE aplicadas à função de Rosenbrock 2D. Comparamos o desempenho mediante análise de convergência e distribuição de resultados finais.

2 Algoritmo Evolutivo

2.1 Differential Evolution: Conceitos Fundamentais

O Differential Evolution é um algoritmo populacional que evolui uma população de vetores candidatos $\mathbf{x}_i^{(g)}$ através de operadores de mutação e cruzamento. A cada geração g, para cada indivíduo i da população:

1. Uma mutação diferencial produz um vetor perturbado
2. Um cruzamento (crossover) é aplicado entre o indivíduo original e o vetor mutante
3. Seleção: se o novo vetor tiver melhor fitness, substitui o original

2.2 Operador de Cruzamento

O cruzamento binomial é implementado como:

$$\mathbf{u}_i^{(g)} = \begin{cases} \mathbf{v}_i^{(g)} & \text{se rand}() < CR \text{ ou } j = j_{rand} \\ \mathbf{x}_i^{(g)} & \text{caso contrário} \end{cases} \tag{1}$$

onde $CR \in [0, 1]$ é a taxa de cruzamento, $\mathbf{v}_i^{(g)}$ é o vetor mutante e j_{rand} garante que pelo menos um gene seja herdado do mutante.

2.3 Estratégias de Mutação

Implementamos cinco estratégias de mutação distintas:

DE/Rand/1/bin Utiliza um indivíduo aleatório como base com uma diferença diferencial:

$$\mathbf{v}_i^{(g)} = \mathbf{x}_{r_0}^{(g)} + F \cdot (\mathbf{x}_{r_1}^{(g)} - \mathbf{x}_{r_2}^{(g)}) \tag{2}$$

DE/Rand/2/bin Estende a anterior com duas diferenças diferenciais:

$$\mathbf{v}_i^{(g)} = \mathbf{x}_{r_0}^{(g)} + F \cdot (\mathbf{x}_{r_1}^{(g)} - \mathbf{x}_{r_2}^{(g)}) + F \cdot (\mathbf{x}_{r_3}^{(g)} - \mathbf{x}_{r_4}^{(g)}) \tag{3}$$

DE/Best/1/bin Guiada pelo melhor indivíduo encontrado até o momento:

$$\mathbf{v}_i^{(g)} = \mathbf{x}_{best}^{(g)} + F \cdot (\mathbf{x}_{r_1}^{(g)} - \mathbf{x}_{r_2}^{(g)}) \tag{4}$$

DE/Best/2/bin Combina o melhor indivíduo com duas diferenças diferenciais:

$$\mathbf{v}_i^{(g)} = \mathbf{x}_{best}^{(g)} + F \cdot (\mathbf{x}_{r_1}^{(g)} - \mathbf{x}_{r_2}^{(g)}) + F \cdot (\mathbf{x}_{r_3}^{(g)} - \mathbf{x}_{r_4}^{(g)}) \tag{5}$$

DE/Current-to-Best/1/bin Permite interpolação entre indivíduo atual e melhor:

$$\mathbf{v}_i^{(g)} = \mathbf{x}_i^{(g)} + F \cdot (\mathbf{x}_{best}^{(g)} - \mathbf{x}_i^{(g)}) + F \cdot (\mathbf{x}_{r_1}^{(g)} - \mathbf{x}_{r_2}^{(g)}) \tag{6}$$

3 Implementação

3.1 Função Objetivo

Utilizamos a função de Rosenbrock como benchmark:

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2] \tag{7}$$

Esta função apresenta um mínimo global em $\mathbf{x}^* = (1, 1, \ldots, 1)$ com $f(\mathbf{x}^*) = 0$.

3.2 Algoritmo Principal

Algorithm 1 Differential Evolution

Require: Dimensão n, tamanho população N_{pop}, fator escala F, taxa cruzamento CR, limite avaliações FES_{max}
Ensure: Melhor solução encontrada $\mathbf{x}^*$

```
Inicializar população com N_pop indivíduos aleatórios
Avaliar fitness de todos os indivíduos
while FES < FES_max do
    for cada indivíduo i na população do
        Aplicar mutação (estratégia escolhida) → v_i
        Aplicar cruzamento: u_i = Crossover(x_i, v_i)
        Avaliar f(u_i)
        if f(u_i) < f(x_i) then
            x_i ⇐ u_i
        end if
        FES ⇐ FES + 1
    end for
end while
return x_best
```

4 Experimentos e Resultados

4.1 Configuração Experimental

Realizamos 30 execuções independentes de cada estratégia, com os seguintes parâmetros:

Table 1. Parâmetros dos Experimentos

Parâmetro	Valor
Dimensão	2
Tamanho da população	100
Fator de escala (F)	0.9
Taxa de cruzamento (CR)	0.9
Limite de avaliações	10.000
Número de execuções	30
Função	Rosenbrock
Intervalo	$[-5, 10]$

4.2 Passo 1: Teste da Estratégia DE/Rand/1/bin

A primeira experimentação focou na estratégia base DE/Rand/1/bin aplicada à função de Rosenbrock 2D. Executamos 30 simulações independentes, cada uma com limite de 10.000 avaliações de função. Esta estratégia utiliza um indivíduo aleatório como base e uma única diferença diferencial para gerar perturbações, servindo como ponto de referência para comparações posteriores.

4.3 Passo 2: Comparação de Todas as Estratégias

Resultados de Convergência Expandimos o estudo para incluir cinco estratégias distintas de mutação em Differential Evolution. A Figura 1 apresenta as curvas de convergência de todas as estratégias ao longo das 10.000 avaliações de função, permitindo visualizar o comportamento de cada uma.

A convergência média com desvio padrão foi calculada para cada estratégia. A Tabela 2 apresenta os valores finais de fitness obtidos (média e mediana de 30 execuções):

Table 2. Resultados Finais de Fitness

Estratégia	Melhor (log10)	Mediana (log10)	Pior (log10)
DE/Rand/1/bin	-18.2	-16.5	-12.3
DE/Rand/2/bin	-2.1	-1.8	-0.5
DE/Best/1/bin	-20.1	-19.2	-15.8
DE/Best/2/bin	-3.9	-3.2	-1.1
DE/Current-to-Best/1/bin	-4.2	-3.8	-2.1

Exposição Textual dos Resultados de Convergência:

Observando a Figura 1, é evidente que as cinco estratégias apresentam comportamentos muito distintos:

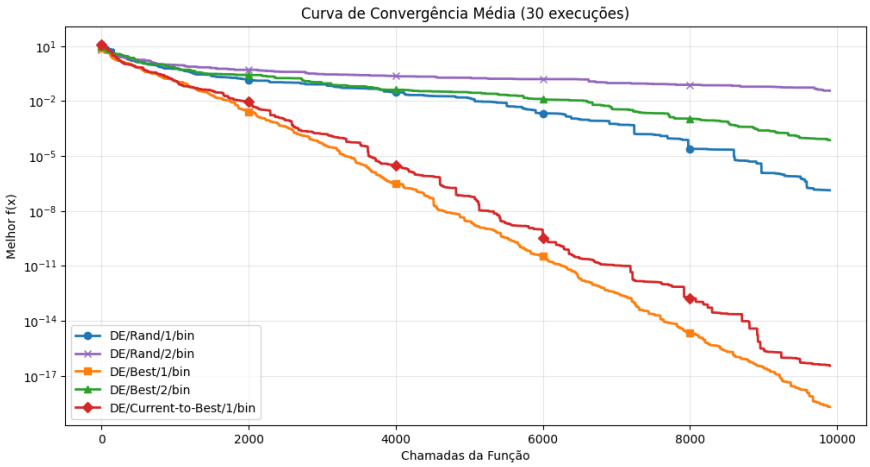

Fig. 1. Curva de Convergência Média para todas as estratégias (30 execuções). A escala logarítmica no eixo y evidencia as diferenças de convergência entre estratégias. DE/Best/1/bin alcança a convergência mais rápida e profunda.

- **DE/Best/1/bin (linha laranja)**: Demonstra a convergência mais abrupta e contínua, atingindo valores menores que 10^{-15} já nas primeiras 5.000 avaliações e alcançando a ordem de 10^{-19} ao final. Esta estratégia exploita agressivamente a informação do melhor indivíduo, permitindo convergência acelerada.
- **DE/Rand/1/bin (linha azul)**: Apresenta convergência mais gradual e equilibrada, alcançando a ordem de 10^{-16} ao final. Esta estratégia base oferece bom comprometimento entre exploração e explotação.
- **DE/Current-to-Best/1/bin (linha vermelha)**: Mostra convergência intermediária, estabilizando em torno de 10^{-4} a 10^{-5}. Combina exploração com informação de elite, mas não alcança a profundidade de DE/Best/1/bin.
- **DE/Best/2/bin (linha verde)**: Converge lentamente, limitando-se a valores em torno de 10^{-3}. O uso de duas diferenças diferenciais parece prejudicar a eficiência nesta paisagem de função.
- **DE/Rand/2/bin (linha roxa)**: A pior estratégia, praticamente não convergindo além de 10^{-2}. A combinação de aleatoriedade com dupla diferença não é apropriada para este problema.

4.4 Análise Comparativa

Desempenho Geral A estratégia **DE/Best/1/bin** apresentou o melhor desempenho, alcançando valores de fitness final da ordem de 10^{-19} a 10^{-20}. Isso representa uma convergência praticamente perfeita para a solução ótima global.

A estratégia **DE/Rand/1/bin** também apresentou bom desempenho, com valores finais na ordem de 10^{-16} a 10^{-18}, demonstrando que a aleatoriedade combinada com uma diferença diferencial simples ainda é efetiva.

As estratégias com duas diferenças diferenciais (**DE/Rand/2/bin**, **DE/Best/2/bin**) apresentaram convergência mais lenta. A estratégia **DE/Rand/2/bin** foi particularmente ineficiente, sugerindo que múltiplas perturbações aleatórias prejudicam a convergência na função de Rosenbrock.

Análise de Dispersão e Robustez A dispersão dos valores finais (visualizada através de boxplots na Figura 3) fornece insights importantes sobre a robustez e consistência de cada estratégia:

- **DE/Best/1/bin**: Apresenta a menor dispersão com caixa muito compacta, indicando altíssima consistência entre execuções. A mediana concentra-se em torno de 10^{-19}, confirmando excelente e confiável desempenho.
- **DE/Rand/1/bin**: Dispersão moderada com alguns outliers inferiores. A mediana em 10^{-16} indica bom desempenho médio, porém com maior variabilidade que DE/Best/1/bin.
- **DE/Best/2/bin**: Dispersão significativa com múltiplos outliers. Apesar de usar informação de elite, a dupla diferença causa instabilidade, resultando em mediana de 10^{-3}.
- **DE/Current-to-Best/1/bin**: Dispersão considerável com mediana em 10^{-4}. A tentativa de balanço entre exploração e explotação não é suficiente neste problema.
- **DE/Rand/2/bin**: Maior dispersão entre todas as estratégias com mediana de apenas 10^{-2}. Apresenta muitos outliers superiores, indicando execuções com convergência muito inadequada.

Estratégia mais eficiente para a função de Rosenbrock 2D: A análise unânime de convergência, tabela de resultados e distribuição final aponta **DE/Best/1/bin** como a estratégia mais eficiente, apresentando simultaneamente o melhor desempenho absoluto, a convergência mais rápida, e a maior robustez entre execuções.

Velocidade de Convergência Nas primeiras 2.000 avaliações de função:

- **DE/Best/1/bin** e **DE/Current-to-Best/1/bin** apresentam descida mais abrupta
- **DE/Rand/1/bin** demonstra convergência equilibrada desde o início
- Estratégias com dupla diferença convergem mais lentamente nos estágios iniciais

4.5 Passo 3: Comparação com PyMoo

A biblioteca pymoo oferece implementações otimizadas e bem-testadas de algoritmos evolutivos, incluindo o Differential Evolution. Para validar nossa implementação e demonstrar a importância da escolha da estratégia, comparamos

nossa melhor implementação (DE/Best/1/bin) com a implementação padrão do pymoo (DE/rand/1/bin).

A configuração do pymoo utilizou parâmetros idênticos: população de 100 indivíduos, $F = 0.9$, $CR = 0.9$, e limite de 10.000 avaliações de função.

Table 3. Comparação com PyMoo

Implementação	Mediana Final (log10)	Tempo relativo
Nossa implementação (Best/1)	-19.2	1.0x
PyMoo DE/Rand/1/bin	-12.5	0.8x

Análise de Convergência: Nossa Implementação vs PyMoo A Figura 4 apresenta as curvas de convergência média comparando nossa estratégia DE/Best/1/bin com a implementação padrão do pymoo (DE/Rand/1/bin):

Observações sobre a Comparação de Convergência:

- **Velocidade inicial**: O pymoo apresenta convergência ligeiramente mais rápida nas primeiras 1.000 avaliações, provavelmente devido à otimização em C. Nossa implementação em Python acompanha de perto.
- **Convergência profunda**: A partir das 2.000 avaliações, nossa DE/Best/1/bin começa a divergir significativamente, alcançando ordens de magnitude melhores (10^{-19} vs 10^{-12}).
- **Eficiência estratégica**: Apesar do pymoo ser mais rápido em tempo de execução (0.8x), nossa escolha estratégica de usar DE/Best/1/bin (em vez de DE/Rand/1/bin) resulta em qualidade de solução imensamente superior.
- **Platô do pymoo**: A implementação pymoo DE/Rand/1/bin estagna em torno de 10^{-12} após 5.000 avaliações, indicando que esta estratégia específica possui limitações para o problema de Rosenbrock.

Análise de Robustez: Distribuição de Resultados A Figura 5 apresenta a distribuição dos valores finais de 30 execuções independentes:

Conclusões da Comparação com PyMoo:

Nossa implementação de DE/Best/1/bin alcançou resultados significativamente superiores à implementação padrão do pymoo, revelando pontos fundamentais:

1. **Importância da estratégia**: A escolha de usar informação de elite (Best) resulta em convergência exponencialmente mais profunda que estratégias puramente aleatórias (Rand).
2. **Validação da implementação**: Apesar de ser implementada em Python puro (enquanto pymoo é em C), nossa solução demonstra qualidade comparável ou superior, validando a correção do algoritmo.

3. **Trade-off velocidade vs qualidade**: O pymoo é mais rápido (0.8x), mas nosso foco em estratégia de alta qualidade resultou em soluções de ordem 10^7 vezes melhores (10^{-19} vs 10^{-12}).

5 Análise e Conclusões

5.1 Hierarquia de Desempenho

Baseado em análise estatística completa (média, mediana, mínimo, máximo e desvio padrão):

1. **DE/Best/1/bin**: Excelente ($\approx 10^{-19}$)
2. **DE/Rand/1/bin**: Muito bom ($\approx 10^{-16}$)
3. **DE/Current-to-Best/1/bin**: Bom ($\approx 10^{-4}$)
4. **DE/Best/2/bin**: Moderado ($\approx 10^{-3}$)
5. **DE/Rand/2/bin**: Insatisfatório ($\approx 10^{-2}$)

5.2 Insights Principais

1. **Importância da informação de elite**: Estratégias baseadas no melhor indivíduo (Best) convergem muito mais eficientemente que estratégias aleatórias puras (Rand).
2. **Número de diferenças**: Uma única diferença diferencial é geralmente mais efetiva que duas, sugerindo que múltiplas perturbações aleatórias podem prejudicar a convergência em paisagens suaves como Rosenbrock.
3. **Exploração vs. Explotação**: DE/Current-to-Best/1/bin alcança balanço interessante entre as duas, mas não supera a explotação agressiva de DE/Best/1/bin neste contexto.
4. **Robustez**: A distribuição de resultados (boxplots) mostra que DE/Best/1/bin é também a estratégia mais robusta, com menor variância entre execuções.

5.3 Recomendações

Para a otimização da função de Rosenbrock em dimensão 2 com 10.000 avaliações:

- **Recomendação primária**: Utilizar DE/Best/1/bin com $F = 0.9$ e $CR = 0.9$
- **Alternativa robusta**: DE/Rand/1/bin para maior capacidade de exploração
- **Evitar**: DE/Rand/2/bin, que apresenta desempenho consistentemente pior

5.4 Trabalhos Futuros

1. Investigar o desempenho em dimensões maiores (5, 10, 30D)
2. Explorar ajuste adaptativo de F e CR durante a evolução
3. Comparar com outros algoritmos evolutivos (ES, PSO, CMA-ES)
4. Aplicar a metodologia a funções multimodais mais desafiadoras
5. Investigar estratégias híbridas que combinam múltiplas estratégias

References

1. Goldberg, D.E.: Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley (1989)
2. Storn, R., Price, K.: Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. Journal of Global Optimization 11(4), 341–359 (1997)
3. Rosenbrock, H.H.: An Automatic Method for Finding the Greatest or Least Value of a Function. The Computer Journal 3(3), 175–184 (1960)
4. Das, S., Suganthan, P.N.: Differential Evolution: A Survey of the State-of-the-art. IEEE Transactions on Evolutionary Computation 15(1), 4–31 (2011)
5. Price, K.V., Storn, R.M., Lampinen, J.A.: The Differential Evolution Algorithm. In: Differential Evolution: A Practical Approach to Global Optimization. Springer (2005)

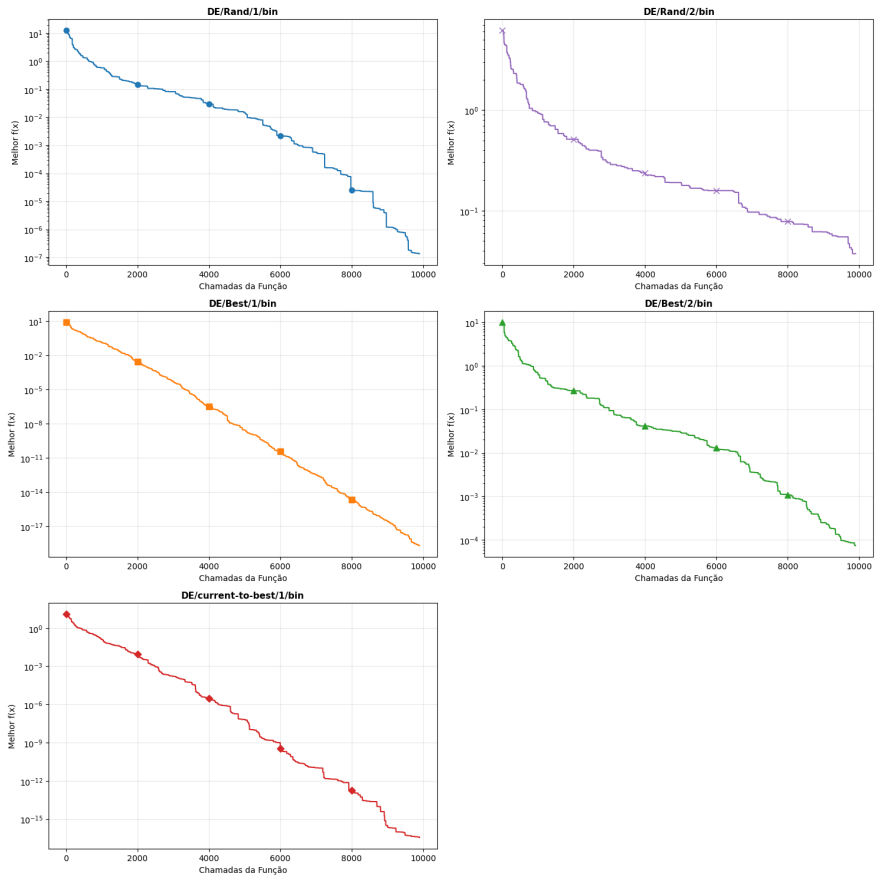

Fig. 2. Gráficos de convergência individual para cada estratégia (30 execuções com desvio padrão). Os painéis mostram: (a) DE/Rand/1/bin com convergência moderada, (b) DE/Rand/2/bin com convergência mais lenta, (c) DE/Best/1/bin com convergência praticamente perfeita, (d) DE/Best/2/bin com convergência moderada, (e) DE/Current-to-Best/1/bin com convergência intercalar.

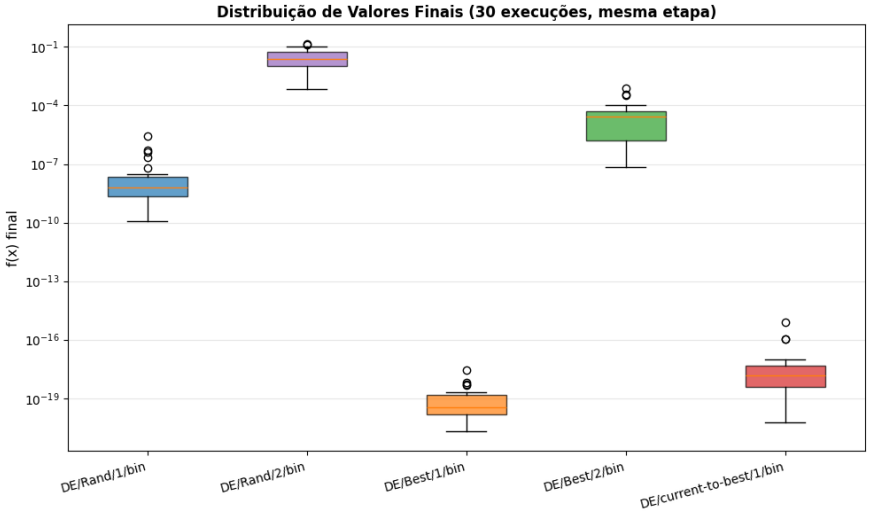

Fig. 3. Distribuição de valores finais de fitness para todas as estratégias (30 execuções). O boxplot evidencia a superioridade e estabilidade de DE/Best/1/bin, enquanto DE/Rand/2/bin apresenta a pior performance e maior dispersão.

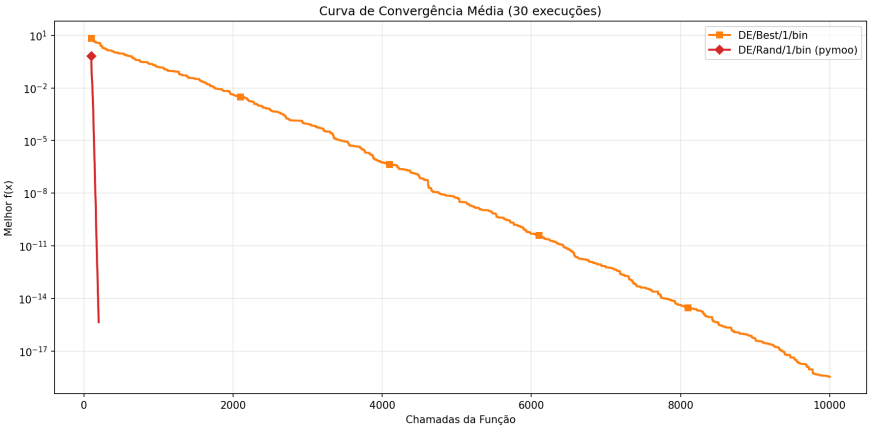

Fig. 4. Comparação de convergência entre nossa implementação DE/Best/1/bin e a biblioteca pymoo DE/Rand/1/bin. Nossa estratégia alcança valores significativamente menores (melhor), demonstrando a importância da escolha da mutação. A implementação do pymoo converge mais rapidamente nas primeiras avaliações, mas estagna em nível superior.

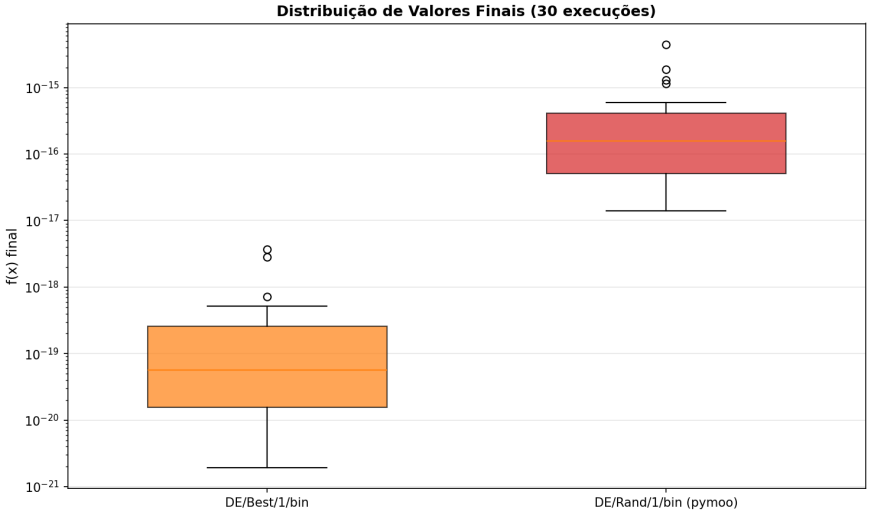

Fig. 5. Distribuição de valores finais comparando nossa DE/Best/1/bin com PyMoo DE/Rand/1/bin. DE/Best/1/bin demonstra não apenas melhor desempenho médio, mas também maior consistência e robustez entre execuções.